

Documentacion tecnica del codigo

Futsi Mini ERP - React/Vite/Tailwind + Django REST Framework

Actualizada al 27 de mayo de 2026.

1. Resumen tecnico

El proyecto esta dividido en dos aplicaciones: backend Django en **back/** y frontend React en **front/**. La API expone recursos REST con Django REST Framework y autenticacion por Token. El frontend consume la API con fetch, renderiza portales por rol y usa Tailwind para la interfaz minimalista.

Estructura principal

Ruta	Proposito
back/futsi_api/settings.py	Configuracion Django: apps, DB SQLite/PostgreSQL, CORS, REST Framework, timezone, media/static.
back/futsi_api/urls.py	Router REST y endpoints auth/login/logout/me.
back/core/models.py	Modelo de dominio: usuarios, sedes, alumnos, asistencia, cargos, pagos, descuentos, gastos, coach logs, cierres y auditoria.
back/core/serializers.py	Serializacion DRF, calculo de saldos, estados de cargos y simulacion de pagos.
back/core/views.py	ViewSets, filtros por rol, acciones de cierre, confirmacion de efectivo, webhook simulado y aprobaciones.
back/core/permissions.py	Permisos por rol: admin, operaciones, cajero, coach y representante.
back/core/management/commands/s eed_demo.py	Carga reproducible de datos demo y usuarios de prueba.
front/src/main.tsx	Aplicacion React completa: login, portales, dashboards, formularios, mapas, graficas y export Excel.
front/src/styles.css	Entrada Tailwind y estilos globales.
tools/*.py	Generadores de documentacion, ERD, Gantt y PDFs.

2. Stack y dependencias

Capa	Tecnologia	Notas
Frontend	React 19, Vite 7, TypeScript, Tailwind 4	SPA con portales por rol y componentes locales.
Mapas	Leaflet + OpenStreetMap	Mapa real de sedes con coordenadas.
Iconos	lucide-react	Botones y acciones.
Backend	Django 5.2, DRF, django-cors-headers	API REST con auth token.
Base local	SQLite	Solo desarrollo/demo.
Base produccion	PostgreSQL	Ya soportado por variables en settings.py.
Docs	reportlab, python-docx, Pillow	Generacion de PDFs, DOCX e imagenes.

3. Configuracion backend

Variable	Uso	Ejemplo
DJANGO_SECRET_KEY	Clave secreta Django.	valor largo generado
DJANGO_DEBUG	Activar/desactivar debug.	false en produccion
DJANGO_ALLOWED_HOSTS	Hosts permitidos.	api.midominio.com
CORS_ALLOWED_ORIGINS	Origenes frontend permitidos.	https://app.midominio.com
DB_ENGINE	Selecciona sqlite o postgres.	postgres
POSTGRES_DB	Nombre de base.	futsi
POSTGRES_USER	Usuario BD.	futsi_app
POSTGRES_PASSWORD	Password BD.	secreto
POSTGRES_HOST	Host BD.	db.internal
POSTGRES_PORT	Puerto BD.	5432

4. Modelo de dominio

- User extiende AbstractUser y agrega role, primary_site, phone, avatar_url, coach_group_name y coach_hourly_rate.
- Site representa sedes con coordenadas, estado activo y ventana de edicion.
- Guardian representa padre/tutor/representante y genera virtual_clabe si falta.
- Student guarda datos operativos y sensibles: foto, responsiva, medico, uniforme y pausas.
- Tournament, Team, Player y Round preparan torneos adultos y jornadas.
- AttendanceSession y AttendanceRecord capturan asistencia y si existia adeudo.
- Charge, Payment y Discount modelan cobranza, pagos parciales, descuentos y conciliacion simulada.
- Expense modela gastos por sede y aprobacion.
- CoachWorkLog captura horas de coach con tarifa snapshot.
- DailyClosure y AuditLog preparan cierres y auditoria.

5. API REST

Endpoint	Recurso	Notas de permiso
/api/auth/login/	Login	Publico, devuelve token y usuario.
/api/auth/me/	Perfil actual	GET/PATCH para contacto y perfil representante.
/api/users/	Usuarios	Solo admin/owner.
/api/sites/	Sedes	Lectura autenticada; escritura admin/owner.
/api/students/	Alumnos	Filtra guardian, cajero y coach por alcance.
/api/attendance-sessions/	Sesiones	Operaciones y coach.
/api/attendance-records/	Asistencia	Operaciones, coach y lectura guardian.
/api/charges/	Cargos	Operaciones; lectura cajero/guardian acotada.
/api/payments/	Pagos	Operaciones, cajero y guardian para acciones permitidas.

Endpoint	Recurso	Notas de permiso
/api/payments/{id}/confirm-cash/	Confirmar efectivo	Representante propio u operaciones.
/api/payments/{id}/simulate-webhook/	Confirmar SPEI/link demo	Simulacion de proveedor.
/api/discounts/	Descuentos	Operaciones; guardian lectura acotada.
/api/expenses/	Gastos	Operaciones.
/api/coach-work-logs/	Horas coach	Operaciones y coach; coach solo sus registros.

6. Logica de cobranza

- charge_balance calcula monto pendiente restando pagos registrados/conciliados y descuentos aprobados.
- sync_charge_status marca paid, partial o pending segun saldo.
- PaymentSerializer asigna site/student/team desde el cargo para evitar capturas inconsistentes.
- Transferencia queda en processing, referencia CLABE y expiracion 72 horas.
- Efectivo queda awaiting_confirmation hasta aceptacion del representante.
- Tarjeta terminal queda registered automaticamente en demo.
- Link de tarjeta queda processing y puede simularse con webhook.
- Los pagos en processing no reducen saldo hasta confirmarse.

7. Frontend

Componente / funcion	Responsabilidad
App	Carga usuario, decide portal segun rol y administra token/data.
LoginScreen	Formulario de acceso.
DashboardPanel	Vista admin/direccion con metricas, mapa, graficas y alertas.
AccountingPortal	Vista contador con grafica X/Y y export Excel.
CashierPortal	Vista cajero para procesar pagos por metodo.
GuardianPortal	Vista representante: adeudos, CLABE, links y efectivo.
CoachPortal	Vista coach: formacion, banca, asistencia y horas.
AttendancePanel	Pase de lista operativo por sede/grupo.
BillingPanel	Cargos, pagos, descuentos y aprobaciones.
ExpensesPanel	Captura y aprobacion de gastos.
StudentsPanel	Alta/edicion/filtros de alumnos y datos sensibles.
exportAccountingWorkbook	Genera Excel XML descargable desde navegador.

8. Comandos de desarrollo

Accion	Comando
Instalar backend	cd back; python -m venv .venv; .venv\Scripts\pip install -r requirements.txt
Migrar backend	cd back; .venv\Scripts\python manage.py migrate
Seed demo	cd back; .venv\Scripts\python manage.py seed_demo --reset
Correr API	cd back; .venv\Scripts\python manage.py runserver 127.0.0.1:8000
Instalar frontend	cd front; npm install
Correr frontend	cd front; npm run dev -- --host 127.0.0.1
Build frontend	cd front; npm run build
Tests backend	cd back; .venv\Scripts\python manage.py test

9. Datos demo y credenciales

Rol	Usuario	Password
Admin	admin	admin12345
Contador	contador	demo12345
Coordinador Roma	coordinador.roma	demo12345
Cajero Roma	caja.roma	demo12345
Cajero Coyoacan	caja.coyoacan	demo12345
Coach Roma	coach.roma	demo12345
Representante	padre.laura / padre.roberto / padre.daniela / padre.jorge	familia12345

10. Testing actual

- Flujo integral: asistencia + pago + descuento + gasto.
- Representante solo ve sus alumnos y no puede crear cargos.
- Representante puede editar datos de contacto/perfil.
- Campos de control del alumno son editables.
- Cajero solo ve su sede y puede procesar pagos.
- Simulacion de transferencia, efectivo y webhook/link.
- Coach solo ve su grupo y puede registrar asistencia/horas sin crear gastos.

11. Deuda tecnica y Sprint 2 tecnico

Tema	Necesidad
Auditoria	Automatizar AuditLog con signals o capa de servicio para pagos, descuentos, gastos, alumnos y cierres.
Settings produccion	Separar settings dev/prod o endurecer por env vars.
Archivos	Mover media local a S3 compatible con URLs privadas/firmadas.
Pagos	Integrar proveedor real y validar firma/idempotencia de webhooks.
Mora	Job programado para generar penalizaciones y avisos.
Importacion	Comando o pantalla de carga Excel con validaciones y reporte de errores.
Permisos	Endurecer escrituras por rol y agregar pruebas por endpoint.
Frontend	Separar componentes grandes de main.tsx a modulos por portal.
Observabilidad	Logs estructurados, Sentry o similar y health checks.

12. Recomendacion de refactor

- 1 Separar front/src/main.tsx en modules: api.ts, types.ts, portals/, panels/, components/.
- 2 Crear servicios backend para pagos, descuentos, auditoria y cierres.
- 3 Agregar permisos por objeto donde haya mas datos reales.

- 4 Crear OpenAPI/Swagger para documentar endpoints.
- 5 Agregar factories/fixtures de prueba para no depender solo de seed_demo.